



University of Global Village (UGV) Department of Computer Science and Engineering

Access Specifiers in Java

Controlling visibility, protecting data, and building secure, well-designed classes

Public · Private · Protected · Default

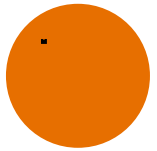
Course Teacher

MD. MEHADI HASAN

Lecturer

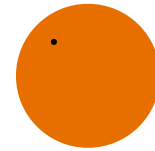
Dept. of CSE, UGV

What We'll Cover



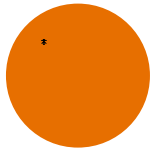
Access Specifiers — Theory

What they are, why they matter, and the 4 types



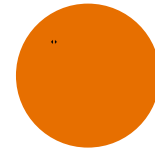
Public, Private, Protected, Default

Definitions, code examples, real-life analogies



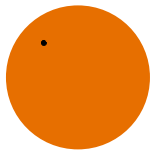
Encapsulation — Concept

Why we need it and how to implement it correctly



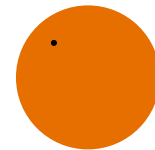
Worked Examples

Without vs. with encapsulation, complete Student class



Advantages & Comparisons

Encapsulation benefits and specifier vs. encapsulation



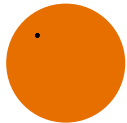
Exam-Ready Questions

Viva questions and 5-mark OBE-based questions

What Are Access Specifiers?

Access Specifiers (also called Access Modifiers) are keywords in Java that determine **who can access a class, variable, method, or constructor**. They are mainly used to control the visibility of program elements and to protect data from unauthorized access. In simple terms, an access specifier answers the question: **"Who is allowed to see or use this class/method/variable?"**

THINK OF IT LIKE A UNIVERSITY SECURITY SYSTEM



Public

Anyone can enter the campus



Protected

Only students & teachers of the university



Default

Only people within the same department



Private

Only the owner (the class itself)

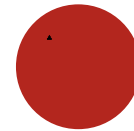
Why Do We Need Access Specifiers?

Without access specifiers, every variable and method could be accessed and modified by anyone. That leads to:



Data hiding

Prevent outside code from directly modifying sensitive data (e.g., a bank balance).



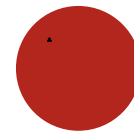
Controlled access

Force other classes to interact with your data only through defined methods (getters/setters), which can include validation logic.



Modularity

Keep internal implementation details hidden while exposing only what's necessary — this makes code easier to maintain and change without breaking other parts of the program.



Security

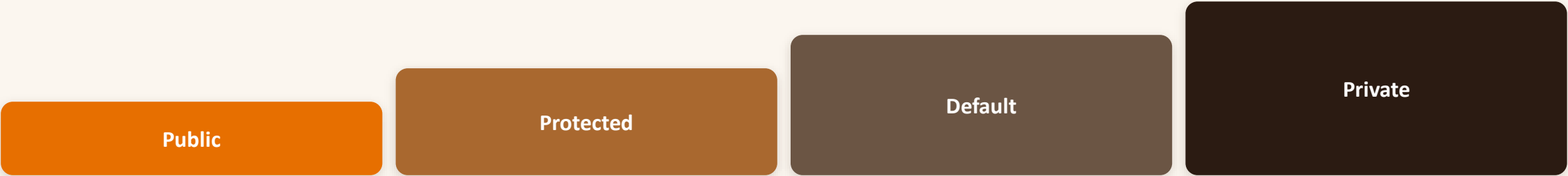
Limit how much of a class's internal workings can be tampered with from outside.

Access specifiers solve these problems by controlling exactly who can see and change what.

Java Provides Four Access Specifiers

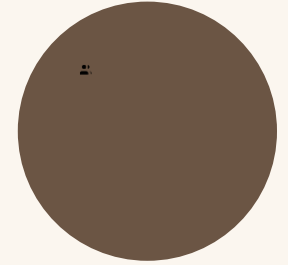
Access Modifier	Same Class	Same Package	Subclass (Diff. Package)	Other Package
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
Default	✓	✓	✗	✗
Private	✓	✗	✗	✗

SECURITY LEVEL, LOW TO HIGH



Public offers the widest access (least secure); Private offers the narrowest access (most secure).

Default Access Specifier



When **no access modifier** is written, Java automatically applies default access — members become accessible only within the same package.

Student.java

```
class Student {  
    String dept = "CSE";    // default access  
  
    void showDept() {  
        System.out.println("Dept: " + dept);  
    }  
}  
  
class Main {    // must be in the SAME package  
    public static void main(String[] args) {  
        Student s = new Student();  
        System.out.println(s.dept); //✓ Works - same package  
    }  
}
```

EXPLANATION

- String department has default access since no modifier is used
Another class in the same package can reach it directly
A class from a different package cannot access it

REAL-LIFE EXAMPLE

A department notice board — only students of that department can see it. Students from another department cannot.

Public Access Specifier

A **public** member can be accessed from any class, any package, and anywhere in the program.

When methods, variables, classes, and so on are declared **public**, then we can access them from anywhere. The public access modifier has no scope restriction.

Main.java

```
class Student {  
    public String name = "Rahim";  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        System.out.println(s.name);  
    }  
}
```

// Output: Rahim

EXPLANATION

- The variable name is declared public, so it can be read from outside the Student class

In main(), s.name is accessed directly from the Main class without any error

Public members need no getter method to be reached from elsewhere

Public — Real Life & Trade-offs

REAL-LIFE EXAMPLE

Imagine a university website — everyone can freely see basic information:

University name

Location

Contact number



ADVANTAGES

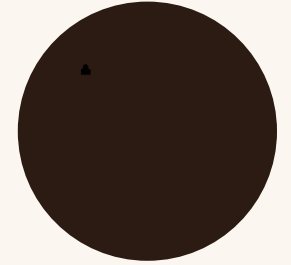
- Easy to access
- Useful for common methods
Can be used anywhere in the program



DISADVANTAGES

- Less secure — open to anyone
- Any class can modify public variables

Private Access Specifier



A **private** member can only be accessed inside the same class. No other class can access it directly. When variables and methods are declared private, they cannot be accessed outside of the class.

Main.java

```
class Student {
    private int age = 22;

    public void showAge() {
        System.out.println(age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.showAge();
        // s.age; --> Error!
    }
}
```

EXPLANATION

- private int age cannot be accessed outside the Student class

The line s.age; is illegal because age is private

Instead, we reach it indirectly through the public method showAge()

Private — Real Life & Trade-offs

REAL-LIFE EXAMPLE

Think of your ATM PIN — only you know it, and nobody else can access it directly. This mirrors a private variable.

Known only to the owner

No direct outside access

Reached only through a controlled process



ADVANTAGES

- Highest security of all specifiers
- Prevents accidental modification
- Directly supports encapsulation

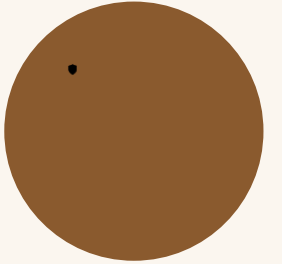


DISADVANTAGES

- Cannot be accessed directly outside the class
- Requires getter and setter methods to expose data

Protected Access Specifier

A **protected** member can be accessed inside the same package, and by subclasses (child classes) even in a different package. When methods and data members are declared `protected`, we can access them within the same package as well as from subclasses. *(via inheritance).*



```
Main.java
class Animal {
    protected void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    public void bark() {
        sound();
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.bark();
    }
}
```

EXPLANATION

- protected void sound() is defined in the parent class Animal
- Dog extends Animal, so it inherits access to sound()
- Calling sound() inside bark() works because of the protected inheritance link

Protected — Real Life & Trade-offs

REAL-LIFE EXAMPLE

A teacher's office — only teachers and department members can enter; not everyone on campus.

Open to same-package members

Open to subclasses elsewhere

Closed to unrelated outside classes



ADVANTAGES

- Supports inheritance between classes
- Allows controlled, purposeful access



DISADVANTAGES

- Less secure than private — reachable through inheritance

Summary of Access Specifiers

Modifier	Accessibility	Security Level
Public	Everywhere	Lowest
Protected	Same package + child class	Medium
Default	Same package	High
Private	Same class only	Highest

In short: the more restrictive the specifier, the safer your data — but the less flexible your access.

Important Questions — 5 Marks (OBE Based)

C2 · Understand

CLO: Explain OOP principles & Java language features

Q1

Explain the four access specifiers in Java. Compare their accessibility levels and identify the situations where each access specifier should be used.

C3 · Apply

CLO: Apply OOP concepts to design & implement Java applications

Q2

Develop a Java program that demonstrates the use of **public**, **private**, **protected**, and **default** access specifiers in appropriate classes. Explain the output.

C4 · Apply

CLO: Analyze design trade-offs in object-oriented Java code

Q3

Design a Java class named **Student** that uses different access specifiers for its data members and methods. Justify your choice of each access specifier based on accessibility and security requirements.



Thank You

Access Specifiers · Encapsulation · Java OOP Fundamentals